

O Especialista em Modelos Locais e o Dev que Está Sangrando na Fatura da API

Transcrição da prática 1 · 15 de setembro de 2026, 15:55

Téo Albuquerque 00:00

"Cara, eu amo o Claude Code, mas tô sangrando uns cem dólares por mês na API. Me disseram que dá pra rodar ele de graça com o Ollama, local.

Mas eu fico de pé atrás: se é grátis e roda no meu notebook, qual é a pegadinha? Vai ser um lixo perto do Claude de verdade? E meu cliente exige que o código não saia da máquina — isso resolve a parte de privacidade?"

01:13 Você

Sim, rodar o Claude Code via Ollama resolve totalmente a privacidade do seu cliente, pois todo o processamento passa a ser 100% local. [1] (<https://elisaterumi.substack.com/p/como-rodar-claude-code-com-ollama>)

No entanto, a grande "pegadinha" é que você não estará rodando o modelo Claude da Anthropic, mas sim a ferramenta CLI (a interface de terminal) conectada a um modelo de código aberto (como o Qwen ou Llama). [1] (https://www.instagram.com/p/DW_-WVWjQ9P/), [2] (<https://www.remissaonline.com.br/blog/como-usar-claude-code-de-graca/>)

Para entender se vale a pena trocar a conveniência da API paga pela infraestrutura local, veja abaixo os prós, os contras e as respostas para os seus receios.

A Pegadinha Desmascarada

O Claude Code é composto por duas partes: a interface de terminal (CLI) — que lê seus arquivos, roda testes e executa comandos — e o modelo de IA (o Claude 3.5 Sonnet na nuvem) que toma as decisões. [1] (<https://www.youtube.com/watch?v=jlGmfSfbxYA&t=13>)

Quando você muda para o Ollama, você mantém a interface CLI, mas substitui o cérebro da Anthropic por um modelo open-source rodando no chip do seu notebook.

Téo Albuquerque 01:16

"Tá, entendi o geral, mas deixa eu te interromper rapidinho: tô querendo saber na prática como isso resolve meu problema sem complicar ainda mais a vida. Tipo, como que eu configuro isso no meu setup atual? E, mais importante, meu notebook vai dar conta de rodar um modelo decente ou vou ter que vender um rim pra comprar hardware novo?"

Para saber se o seu notebook vai dar conta ou se você vai precisar de um hardware novo, a regra é clara: o fator decisivo é a memória RAM (e a VRAM da placa de vídeo), não o processador.

Abaixo está o guia prático para você saber exatamente se sua máquina aguenta e como fazer essa transição em 5 minutos.

Seu notebook vai dar conta? (O teste do hardware)

Como os modelos rodam inteiramente na memória, o tamanho do modelo que você pode usar depende de quanta RAM você tem livre. Veja em qual categoria você se encaixa:

— 16 GB de RAM (Limite mínimo): Você consegue rodar modelos de 7 bilhões de parâmetros (7B), como o qwen2.5-coder:7b. Ele é super rápido, consome cerca de 5 GB a 6 GB de RAM e dá conta de tarefas diárias, geração de funções e correção de bugs simples. Seu notebook vai dar conta, mas evite abrir 50 abas no Chrome junto.

— 32 GB de RAM ou Mac M-Series (Ideal): Você consegue rodar modelos de 14 bilhões ou 32 bilhões de parâmetros (14B / 32B). O qwen2.5-coder:14b é o "ponto doce" entre inteligência e velocidade, chegando bem mais perto do Claude original em raciocínio lógico.

— 8 GB de RAM (Venda o rim): Infelizmente, para desenvolvimento com agentes locais, não dá. Você ficaria limitado a modelos de 1B ou 3B de parâmetros, que erram muito código e vão te fazer passar raiva.

Como configurar no seu setup atual (Passo a Passo)

A integração do Claude Code com o Ollama é feita em segundos usando variáveis de ambiente que redirecionam o "cérebro" da CLI para a sua máquina.

1. Deixe o Ollama pronto

Baixe o instalador oficial no site do Ollama e execute-o. Depois, abra o seu terminal e baixe o modelo de código mais eficiente da atualidade rodando:

```
ollama run qwen2.5-coder:7b
```

(Se você tiver 32GB de RAM, mude o final para :14b ou :32b). Deixe o modelo carregar e fazer o primeiro download. Quando ele abrir o chat, você pode digitar /exit para fechar, mas o serviço continuará rodando em segundo plano.

2. Configure as credenciais locais no terminal

Para não complicar a sua vida, você não precisa reinstalar o Claude Code. Você só precisa avisar a ele que, em vez de ir para os servidores da Anthropic, ele deve conversar com o seu Ollama local.

No seu terminal (dentro da pasta do projeto do seu cliente), execute os comandos abaixo:

No Linux / macOS:

```
# Redireciona a API para o seu Ollama local
export ANTHROPIC_BASE_URL="http://localhost:11434/v1"
# Define uma chave visual fictícia (o Claude Code exige que o campo não esteja vazio)
export ANTHROPIC_API_KEY="local-ollama"
# Inicializa o Claude Code no projeto
claude
```

No Windows (PowerShell):

```
$env:ANTHROPIC_BASE_URL="http://localhost:11434/v1"  
$env:ANTHROPIC_API_KEY="local-ollama"  
claude
```

Pronto! O Claude Code vai abrir exatamente com a mesma interface que você já ama, vai escanear seus arquivos locais, mas toda a inteligência e o processamento estarão acontecendo offline na porta 11434 da sua própria máquina. O consumo da API da Anthropic vai zerar na hora.

Téo Albuquerque 03:04

"Beleza, entendi que o hardware é chave aí. Acho que meu notebook tá no limite do 'vai dar conta', já que tem dezesseis gigas de RAM. Mas me explica direito: depois que eu configuro as variáveis de ambiente, eu consigo usar o Claude Code do mesmo jeito que uso hoje? E sério, o modelo tipo qwen ou llama vai dar conta de tarefas mais complexas, tipo implementar algo mais avançado ou ajustar testes automatizados?"

A interface vai ser a mesma?

Sim, a experiência visual e os comandos serão idênticos. [1]

(https://medium.com/@markbabcock_79883/run-claude-code-with-open-source-models-via-ollamas-anthropic-api-compatibility-0eeeb3a415f4), [2] (https://www.linkedin.com/posts/mateusz-klatt_claude-code-with-anthropic-api-compatibility-activity-7419870554540920833-8d9B)

Como o Ollama possui compatibilidade nativa com a API de mensagens da Anthropic, a ferramenta CLI do Claude Code não sabe (e não se importa) que está conversando com a sua máquina em vez da nuvem. [1] (<https://gist.github.com/AUAggy/ccf6df83c297e76191ff2de8eb6a5168>), [2] (<https://ollama.com/blog/claude>)

Você continuará vendo:

- O mesmo terminal interativo.

- O fluxo de permissões pedindo autorização antes de rodar comandos ou editar arquivos.

- Os mesmos comandos integrados, como a execução automática de testes. [1]

(<https://www.mindstudio.ai/blog/ollama-run-ai-models-locally-claude-code-workflows>), [2]

(https://www.linkedin.com/posts/mateusz-klatt_claude-code-with-anthropic-api-compatibility-activity-7419870554540920833-8d9B), [3] (<https://docs.ollama.com/integrations/claude-code>)

O modelo 7B (para 16GB de RAM) dá conta de tarefas avançadas?

Aqui precisamos alinhar a expectativa com a realidade técnica do seu hardware. Com 16 GB de RAM, você fica limitado a modelos de 7 bilhões de parâmetros (7B). [1]

(<https://www.mindstudio.ai/blog/ollama-run-ai-models-locally-claude-code-workflows>)

1. Ajustar testes automatizados: Sim, ele dá conta muito bem.

Mapear um bug em uma rota, ler a resposta do erro no terminal e corrigir a asserção ou o payload em um arquivo de teste é uma tarefa de escopo fechado. O qwen2.5-coder:7b brilha muito nisso porque ele foi treinado extensivamente em sintaxe de testes e estruturas de código. Ele vai ler o teste que quebrou, reescrever o trecho isolado e pedir sua permissão para rodar a suíte de testes de novo. [1] (<https://www.mindstudio.ai/blog/ollama-run-ai-models-locally-claude-code-workflows>)

2. Implementar algo mais avançado / arquitetural: Não, ele vai patinar.

Se por "avançado" você quer dizer criar um microserviço do zero seguindo um padrão de design específico, ou pedir para ele "reescrever esse módulo de pagamento para suportar concorrência e atualizar os outros 4 arquivos que dependem dele", o modelo 7B vai falhar onde o Claude 3.5 Sonnet original acertaria de primeira.

O motivo: Modelos menores perdem a "linha de raciocínio" (coerência lógica) quando precisam processar contextos muito grandes (muitos arquivos abertos ao mesmo tempo) ou lidar com lógica de negócios altamente abstrata.

Téo Albuquerque 04:16

"Ah, saquei! Então vou ter a mesma interface, sem dor de cabeça pra adaptar meu fluxo, mas com os modelos menores, tipo o 7B, ele vai mandar bem só em tarefas mais pontuais. Pra coisas mais pesadas, ele vai tropeçar, né? Faz sentido. Bom, pelo menos já sei que dá pra resolver o básico e garantir a privacidade do cliente. Vou testar isso aí e ver como ele se comporta no meu dia a dia. Valeu pela explicação, foi bem direto ao ponto!"

Ao consolidar essa mudança para o modelo local, você ganha autonomia total sobre o seu ambiente de desenvolvimento. Estes são os três ganhos reais e imediatos que você terá ao rodar o Claude Code com o Ollama no seu notebook de 16 GB de RAM:

- Custo Zero por Token: Você estanca o sangramento de US\$ 100+ por mês. Não importa se você pedir para a IA escanear o projeto inteiro 50 vezes no mesmo dia, gerar milhares de linhas de código ou rodar testes repetidamente em um loop. A sua fatura da API da Anthropic vai zerar, e o seu único custo será a energia elétrica do notebook.
- Privacidade Absoluta (Zero Vazamento): Você cumpre a exigência do seu cliente com 100% de conformidade. Como o tráfego de dados acontece estritamente dentro da porta interna da sua máquina (localhost:11434), nenhum trecho de código, segredo de negócio, chave de API ou propriedade intelectual do projeto é enviado para servidores de terceiros ou usado para treinar modelos na nuvem.
- Funcionamento 100% Offline: Você não depende mais de conexão com a internet para programar com assistência de IA. É possível codar e usar o Claude Code dentro de um avião, no interior, durante instabilidades na rede ou em redes públicas de cafeterias sem se preocupar com segurança. Se você desligar o Wi-Fi do notebook, o assistente continuará respondendo e editando seus arquivos com a mesma velocidade.

Téo Albuquerque 05:36

"Cara, esses três pontos realmente fazem a diferença. Custo zero, privacidade total e ainda poder usar offline? Isso resolve não só a exigência de sigilo do cliente, mas também o peso no bolso. Vou configurar isso logo pra testar e, se o modelo der conta do básico, já me ajuda pra caramba. Valeu pelas dicas, acho que tô pronto pra colocar isso em prática!"

Para que a sua migração seja bem-sucedida, é preciso jogar limpo com a realidade técnica. Mudar para o Ollama traz liberdade e economia, mas exige que você aceite alguns compromissos e limitações reais no seu dia a dia de desenvolvimento.

Estes são os trade-offs que você precisa colocar na balança:

— Abre mão do topo da cadeia alimentar: Você deixará de usar o Claude 3.5 Sonnet na nuvem. Os modelos open-source evoluíram absurdamente, mas nenhum modelo de 7B de parâmetros chega aos pés da capacidade de abstração, lógica complexa e refinamento que a Anthropic entrega nos servidores bilionários dela.

— Dependência brutal do seu Hardware (RAM/VRAM): A velocidade e a inteligência do seu assistente agora têm um teto físico: os seus 16 GB de RAM. Se o modelo local for grande demais para a sua memória, o seu notebook vai travar ou a IA vai responder gerando pouquíssimas palavras por segundo (o famoso gargalo de tokens por segundo).

— A armadilha das Tools e do Contexto: O Claude Code não é um chat comum; ele é um agente. Ele precisa que o modelo local tenha suporte nativo e robusto a tool calling (capacidade da IA decidir usar ferramentas como ler arquivo, rodar comando, etc.) e um bom gerenciamento de contexto. Modelos menores tendem a "esquecer" o que estavam fazendo no meio de uma tarefa longa ou errar a sintaxe dos comandos que o Claude Code tenta executar.

Para quem essa troca é perfeita?

Essa configuração local com o Claude Code é o cenário ideal se o seu foco principal for:

— Tratar códigos altamente sensíveis: Onde o sigilo exigido pelo cliente torna a nuvem uma opção proibitiva.

— Construir MVPs, automações e demos: Validar ideias rapidamente, criar estruturas de software padrão e gerar rotinas repetitivas sem gastar nada.

— Estudo e aprendizado: Entender como agentes de IA funcionam por dentro sem o cronômetro do cartão de crédito rodando em dólares.

Sabendo desses prós e contras, como você quer seguir com a configuração?

— Posso te passar os comandos para salvar as variáveis de ambiente de forma definitiva no seu sistema.

— Podemos configurar as regras de contexto para evitar que o modelo 7B estoure os seus 16 GB de RAM.

Téo Albuquerque 06:44

"Boa, agora ficou bem claro. Então, eu ganho economia e privacidade, mas perco um pouco da potência e da fluidez que tinha com o Claude na nuvem. Faz sentido pra projetos menores e sensíveis, mas vou ter que ficar esperto com o limite da RAM e o contexto. Bora seguir com os comandos pra salvar as variáveis e ajustar pro meu notebook não engasgar. Quero testar logo isso no meu ambiente!"

Excelente escolha. Vamos configurar tudo para que o Claude Code inicialize automaticamente integrado ao seu Ollama local e com o consumo de memória otimizado para os seus 16 GB de RAM.

1. Salvando as variáveis de ambiente de forma definitiva

Para você não ter que digitar os comandos toda vez que abrir o terminal, vamos salvar as configurações no arquivo de perfil do seu sistema. Descubra qual é o seu sistema operacional abaixo e execute os comandos no terminal:

Se você usa Mac ou Linux (Terminal Bash ou Zsh)

A maioria dos Macs modernos usa o zsh. Execute estes comandos para salvar as variáveis no seu arquivo de configuração (se usar Bash, mude `.zshrc` para `.bashrc`):

```
# Adiciona as variáveis de ambiente ao seu perfil do terminal
echo 'export ANTHROPIC_BASE_URL="http://localhost:11434/v1"' >> ~/.zshrc
echo 'export ANTHROPIC_API_KEY="local-ollama"' >> ~/.zshrc
# Atualiza o terminal atual com as novas configurações
source ~/.zshrc
```

Se você usa Windows (PowerShell)

Abra o PowerShell como Administrador e execute os comandos abaixo para salvar as variáveis de forma permanente no escopo do usuário:

```
[System.Environment]::SetEnvironmentVariable('ANTHROPIC_BASE_URL', 'http://localhost:11434/v1', 'User')
[System.Environment]::SetEnvironmentVariable('ANTHROPIC_API_KEY', 'local-ollama', 'User')
```

Nota: Após rodar os comandos no Windows, feche o seu terminal atual e abra um novo para as alterações fazerem efeito.

2. Ajustando o Claude Code para seu notebook não engasgar

Como você está com 16 GB de RAM e usando o `qwen2.5-coder:7b`, o maior risco de lentidão (o famoso "engasgo") acontece quando o Claude Code tenta ler arquivos gigantescos ou históricos de chat muito longos, estourando a memória e forçando o notebook a usar o HD/SSD como memória secundária.

Podemos controlar isso limitando o tamanho do contexto e o comportamento do agente diretamente na inicialização ou no arquivo `.claudecode.json` do seu projeto.

A. Limite o contexto na inicialização

Sempre que for iniciar o Claude Code na pasta do projeto, você pode passar a flag `--max-tokens` para limitar o tamanho máximo das respostas do modelo, economizando processamento:

```
claude --max-tokens 2048
```

B. Crie regras de escopo para o Agente

O Claude Code lê um arquivo chamado `.clauderc` ou instruções customizadas se você configurá-las. Crie um arquivo chamado `claude.auto` ou configure o comportamento inicial dele para evitar abusos:

- Evite buscas globais automáticas: Diga explicitamente no chat para ele focar apenas no arquivo que você quer mexer (Ex: "Analise apenas o arquivo src/auth.js e corrija o bug...").
- Monitore o uso de RAM: Antes de rodar, abra o Gerenciador de Tarefas (Windows) ou o Monitor de Atividade (Mac). O processo ollama_llama_server deve consumir entre 4.5 GB e 6 GB de RAM. Se ele começar a usar mais que isso, reinicie o serviço do Ollama para limpar o cache de contexto.

O Teste de Fogo

Agora que está tudo configurado:

- Certifique-se de que o Ollama está rodando (ollama run qwen2.5-coder:7b).
- Entre na pasta do projeto do seu cliente.
- Digite simplesmente claude e dê Enter.

Se você encontrar qualquer erro de conexão ou comportamento estranho nos primeiros testes, me avise! Qual é o sistema operacional exato (Windows, macOS ou Linux) que você está usando para rodar esse setup?

Téo Albuquerque 08:11

"Show, bora então! Já entendi como salvar as variáveis no meu sistema e limitar o contexto pra não sobrecarregar a máquina. Eu uso macOS, então vou seguir com o terminal e configurar tudo no .zshrc. Vou testar isso agora e ver como ele se comporta com o Ollama rodando. Se der algum erro, te chamo, mas por enquanto, valeu mesmo pela ajuda! Tá bem claro agora."

09:59 Você

Vai dar super certo! Como você está no macOS com o zsh, esse setup costuma rodar de forma muito fluida, especialmente se o seu Mac for da linha Apple Silicon (M1/M2/M3), que gerencia a memória unificada incrivelmente bem para modelos locais.

Só um último lembrete de ouro antes de você rodar o seu teste: se em algum momento você precisar voltar a usar o Claude original na nuvem (pagando a API), basta abrir o terminal e limpar as variáveis temporariamente com o comando unset ANTHROPIC_BASE_URL e unset ANTHROPIC_API_KEY. Quando fechar e abrir o terminal de novo, ele voltará para o modo local definitivo que acabamos de salvar.

Boa sorte com os testes e com o projeto do seu cliente! Se o modelo travar, der erro de conexão ou começar a "alucinar" nos comandos, é só dar um grito aqui.

Téo Albuquerque 10:02

"Ah, boa dica! Esse comando pra voltar pro Claude na nuvem vai ser útil se eu precisar comparar as capacidades depois. Valeu mesmo por todo o suporte, vou rodar os testes agora e ver como ele se comporta no meu projeto. Qualquer coisa, te chamo. Abraço e obrigado mais uma vez!"

No Linux, a lógica é idêntica à do macOS, mas com uma pequena diferença: dependendo da sua distribuição (Ubuntu, Debian, Fedora, Arch, etc.), o terminal padrão pode ser o Bash em vez do Zsh. Veja abaixo como deixar isso definitivo no seu setup Linux:

1. Salvando as variáveis de forma definitiva

Primeiro, descubra qual terminal você está usando rodando `echo $SHELL`. Depois, escolha a opção correspondente abaixo:

Se o seu terminal for o BASH (Padrão na maioria das distros como Ubuntu e Debian):
Execute estes comandos para salvar as variáveis no arquivo `.bashrc`:

```
# Adiciona as variáveis ao final do seu arquivo de configuração do Bash
echo 'export ANTHROPIC_BASE_URL="http://localhost:11434/v1"' >> ~/.bashrc
echo 'export ANTHROPIC_API_KEY="local-ollama"' >> ~/.bashrc
# Recarrega o arquivo para aplicar as alterações no terminal atual
source ~/.bashrc
```

Se o seu terminal for o ZSH (Comum se você usa Manjaro ou customizações como Oh My Zsh):
Caso tenha instalado o Zsh no Linux, o comando é exatamente o mesmo que você usou no Mac:

```
echo 'export ANTHROPIC_BASE_URL="http://localhost:11434/v1"' >> ~/.zshrc
echo 'export ANTHROPIC_API_KEY="local-ollama"' >> ~/.zshrc
source ~/.zshrc
```

2. Dica bônus para Linux: Gerenciando o serviço do Ollama

Ao contrário do Mac, onde o Ollama roda como um app visual na barra de menus, no Linux ele roda nativamente como um serviço do sistema (systemd). Isso te dá um controle muito legal sobre o consumo de hardware:

Ver se o Ollama está ativo e consumindo memória:

```
systemctl status ollama
```

Parar o Ollama quando terminar de programar (para liberar seus 16GB de RAM para outras tarefas pesadas ou jogos):

```
sudo systemctl stop ollama
```

Iniciar o Ollama novamente quando for codar:

```
sudo systemctl start ollama
```

Fora isso, o comportamento do Claude Code e do modelo `qwen2.5-coder:7b` será exatamente o mesmo. Se o seu Linux tiver uma placa de vídeo da Nvidia configurada com os drivers proprietários e o `nvidia-container-toolkit`, o Ollama vai jogar o modelo direto na VRAM da placa, deixando o sistema ainda mais rápido do que rodando na memória RAM comum!

O seu Linux está rodando em uma máquina com placa de vídeo dedicada (Nvidia/AMD) ou você está usando gráficos integrados da Intel/AMD? Posso te ajudar a verificar se o Ollama está extraindo o máximo de aceleração por hardware se precisar.

Téo Albuquerque 11:39

"Entendido! Como eu tô no macOS, não preciso me preocupar com essas diferenças do Linux agora, mas é bom saber disso pra caso eu precise configurar em outro ambiente no futuro. Valeu mesmo por todo o suporte! Vou dar sequência nos testes e já tô confiante de que vai rolar sem problemas. Até a próxima!"

FIM DA SESSÃO

Esta transcrição inclui conteúdo gerado por IA e é fornecida para seus propósitos de aprendizado pessoal.